Assignment 2: Implementing ADTs and Using Data Structures

Description

In this assignment, you'll work in groups to create your own versions of an arraylist, a doubly linked list, a stack and a queue. You'll then use these data structures to create a parser application, which will read and confirm that the text files are properly formatted.

Warning: This project is the largest and most complex assignment in this course, do NOT wait until the last week of the deadline to start!

Equipment and Materials

For this assignment, you will need:

- Eclipse, Java 21 and JUnit 4

Note: If your submission does not meet these requirements, you will receive a grade of zero!

Instructions

This assignment consists of three parts, to be completed **outside** of class time. See the course outline and Brightspace for due dates. Complete this assignment with your assigned group.

Part A: Create and Implement a Parser Application using your own data structures (90 marks)

1. You will implement the following data structures according to the ADTs in the provided interfaces:
    - ArrayList
    - Doubly linked list
    - Stack
    - Queue
2. Using the specifications below, you'll then create an application to parse text files for correctness using these data structures.

Part B: Complete an Evaluation as a Group (5 marks)

After completing your parser application, check your work against the provided marking criteria. Your instructor will refer to your group's self-evaluation when grading the assignment and will

© 2026, Southern Alberta Institute of Technology

provide further feedback and grade adjustments as needed. Your instructor is responsible for awarding the group's final grade.

1. Open the Marking Criteria documents (MarkingCriteria_Assignment2.docx) and save a copy with your group's name.
2. As a group, discuss how well you met each criterion and assign yourselves a mark for each row in the table. You may include a short, point form, explanation for your mark in the Notes column.

   Note: This is an opportunity for you to identify any missing pieces according to the marking criteria, make sure you do this step diligently to avoid losing any marks that is clearly stated in this document.
3. Save this file for submission to Brightspace along with your completed code.

Part C: Complete a Peer Assessment (5 marks)

Each student must also complete a peer assessment of themselves and their group members, which will be used to determine the final calculated grades of this assignment for all group members. Your instructor will provide further submission details in class.

Assignment Specifications

Read the specifications very carefully. If you are uncertain about any of the requirements, discuss it with your instructor.

1. Import the assignment2StartingCode project into Eclipse.
2. Complete the implementation of the class **MyArrayList.java** using the supplied **ListADT.java** and **Iterator.java** interfaces using an array as the underlying data structure to store the elements in this list.

   Note: Do NOT modify the ADTs provided in any way! Your instructor will test with the default ADT files.
3. Thoroughly test your implementation of MyArrayList.java for correct functionality using the set of JUnit tests **MyArrayListTest.java** from the starting code.

   Note: Do NOT modify the tests provided in any way! Your instructor will test with the default test files.
4. Complete the implementation of the class **MyDLL.java** using a linked list as the underlying data structure to store each element in the node class **MyDLLNode.java**, based on the same **ListADT.java** and **Iterator.java** interfaces provided by your instructor. Test your implementation using the set of JUnit tests **MyDLLTest.java.**
5. Complete the implementation of the class **MyStack.java** using the **StackADT.java** and **Iterator.java** interfaces provided by your instructor. Use your **MyArrayList.java**

© 2026, Southern Alberta Institute of Technology

implementation as the underlying data structure to store the elements of the stack. Test your implementation using the set of JUnit tests **MyStackTest.java.**

6. Complete the implementation of the class **MyQueue.java** using the **QueueADT.java** and **Iterator.java** interfaces and **EmptyQueueException.java** provided by your instructor. Use your **MyDLL.java** implementation as the underlying data structure to store the elements of the queue. Test your implementation using the set of JUnit tests **MyQueueTest.java.**

WARNING!

You will be penalized **50% of your final mark for the assignment** if you use the java classes from the java.util.*, or similar packages for the implementations of MyStack, MyQueue, MyArrayList and MyDLL.

The exception is using the Arrays.copyOf() or System.arraycopy() methods to resize your array in MyArrayList, the toArray methods and the Iterator implementations. You are also allowed to use any of the standard exceptions in the Java library such as NullPointerException and NoSuchElementException.

Important: *Move to the next step only when you are satisfied that the data structures above perform properly. Then implement your application as described below.*

7. Create a simplified parser application that will accomplish the following:
    - Read the supplied text file from the command line, one line at a time.
    - Parse for errors in the syntax construction according to the rules below using your MyStack and MyQueue implementations.
    - When parsing is complete, report all lines that are not properly constructed in the order in which the errors occur. (e.g. line numbers)

Parsing Rules

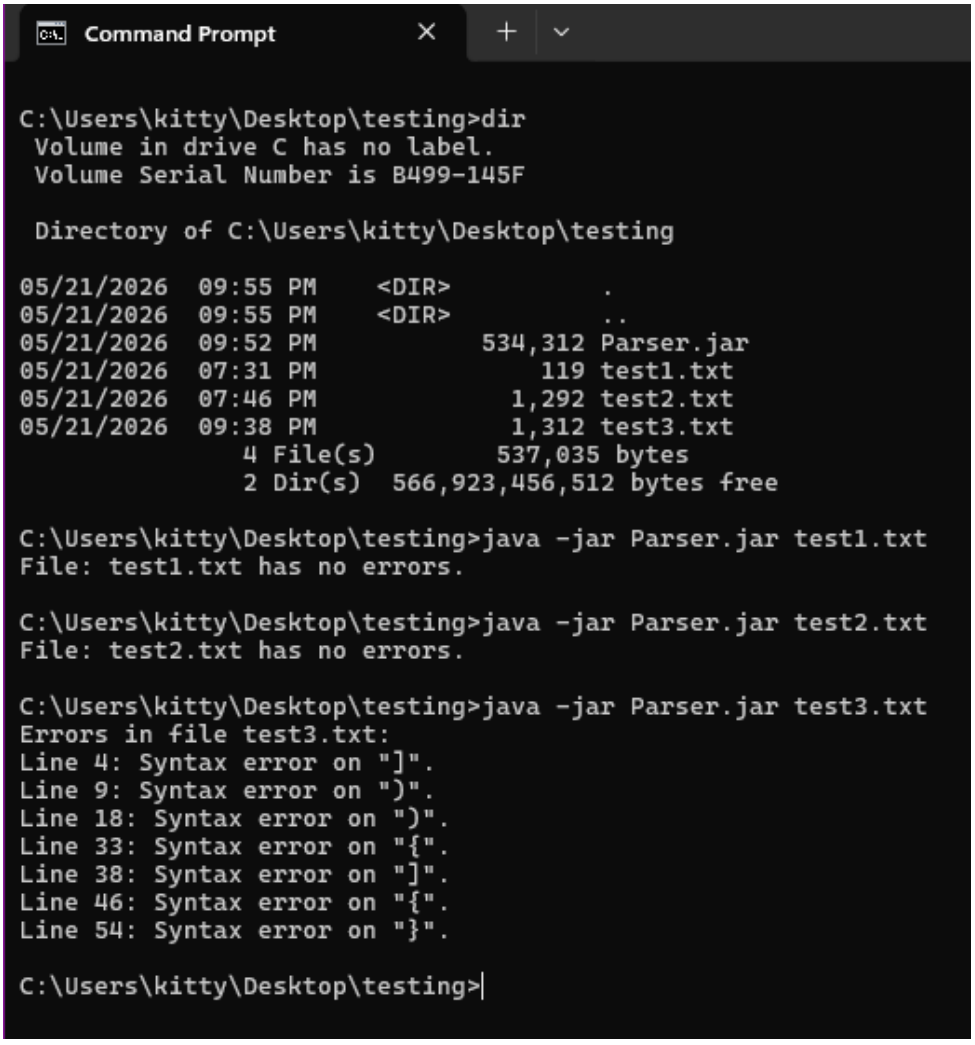
This assignment assumes a restricted set of brackets for simplicity. A text file is syntactically correct if it meets the following requirements:

- Accepted brackets, everything else is ignored for this purpose of this application:
    - `(` `)` – parentheses
    - `{` `}` – curly braces
    - `[` `]` – square brackets
    - `<` `>` - angle brackets

© 2026, Southern Alberta Institute of Technology

- Every opening bracket must have a corresponding closing bracket:
  - A `(` must have a matching `)`
  - A `{` must have a matching `}`
  - A `[` must have a matching `]`
  - A `<` must have a matching `>`
- Normal ordering and nesting rules apply:
  - `( [ ] )` is correct
  - `( [ ) ]` is incorrect
  - `{ [ ( ) ] }` is correct
  - `{ [ ( ] ) }` is incorrect

8. Ensure that your program can be supplied the text file via the command line and show all results of the parsing on the console. E.g.:



© 2026, Southern Alberta Institute of Technology

Note: These images are provided simply to give you an example of what the output would look like – your program output should be similar but does NOT have to match the exact formatting as long as the relevant information are shown. Your instructor will use a different test data file to evaluate your application.

> ***Important:*** *Your errors may differ depending on the logic of your parsing algorithm. Building a "perfect" parser is very difficult! Your application should detect all incorrect bracket formations (i.e. all missing or extra brackets) but the line number reported may not match exactly to the screenshot above.*

Submission Deliverables

Note: This is a group assessment, so only one submission is required per group. If there are multiple submissions, your instructor will only evaluate the last submission. See Brightspace for the exact due date and time.

Your group's submission should include a zipped folder named as the assignment number and your group name (e.g., A2Zelda.zip). The zipped folder should contain the following items:

1) An **executable** Java Archive file (.jar) for your sorting application called **Parser.jar**.
2) A zipped file of the folder containing the complete Eclipse project directory.
    - Using File Explorer, navigate to your project folder inside your Eclipse workspace, right-click and select "Compress to… ZIP file".
3) The completed Marking Criteria document containing your group's evaluation of your application. All group members must be present when completing this document and it acts as your "sign-off" (approval) of the assignment submission.
4) Complete the Peer Evaluation Assessment as instructed by your instructor. Examples of how this evaluation will be used to determine your final grade for this assignment can be found on D2L → Unit 0: Course Information → Individual Grade Calculations for Group Assignments.pdf.

No late assignments or resubmissions will be accepted; you are responsible for the correctness of your deliverables.

© 2026, Southern Alberta Institute of Technology